부록 · 리눅스 명령어

Linux awk · sed · grep · tee 실전 교재

편집기 없이 JSP · conf · properties · YAML · Java · Kotlin 검색/치환/삭제/자동화

1. 핵심 개념

쉘에서 설정파일과 소스코드를 다룰 때 find는 대상 파일 탐색, grep은 문자열·구문 검색, awk는 조건·범위·필드 가공, sed는 치환·삽입·삭제, tee는 화면 출력과 결과 저장에 사용함.

권장 흐름:

```bash
find → grep → 백업 → awk → sed → grep 검증 → tee 보고서 → 필요 시 cron 자동화
```

2. 핵심 명령 비교

명령	역할	대표 기능	예
find	파일 탐색	확장자·경로·조건별 파일 검색	find . -name "*.jsp"
grep	문자열 검색	줄 번호, 재귀, 정규식, 주변 문맥	grep -Rni "ERROR" .
awk	조건/범위 가공	줄 선택, 필드 처리, 집계	awk '/ERROR/{print NR,$0}' app.log
sed	텍스트 편집	치환, 삭제, 삽입, 일괄 수정	sed -i.bak 's/8080/8081/g' file
tee	출력+저장	화면 확인과 파일 기록 동시 수행	grep ERROR app.log | tee errors.txt

3. 편집기 없이 사용하는 실전 명령·예제 35개 이상

1. 문자열과 줄 번호 검색

ERROR가 들어간 줄과 줄 번호를 표시함.

```bash
grep -n "ERROR" app.log
```

2. 대소문자 무시 검색

ERROR, Error, error 등을 모두 찾음.

```bash
grep -ni "error" app.log
```

3. 하위 폴더 전체 검색

현재 경로 아래 모든 파일을 재귀 검색함.

```bash
grep -Rni "ERROR" .
```

4. JSP 파일만 검색

JSP 안에서 session 사용 위치를 찾음.

```bash
grep -Rni --include="*.jsp" "session" .
```

5. Java/Kotlin 동시 검색

Java와 Kotlin 소스의 TODO를 찾음.

```bash
grep -Rni --include="*.java" --include="*.kt" "TODO" .
```

6. 에러 앞뒤 문맥 확인

ERROR 앞뒤 3줄을 함께 표시함.

```bash
grep -n -C 3 "ERROR" app.log
```

7. ERROR/WARN 동시 검색

확장 정규식으로 두 패턴을 동시에 검색함.

```bash
grep -En "ERROR|WARN" app.log
```

8. DEBUG 제외 출력

DEBUG가 없는 줄만 출력함.

```bash
grep -v "DEBUG" app.log
```

9. awk로 ERROR와 줄 번호 출력

NR을 이용해 실제 원본 줄 번호와 내용을 출력함.

```bash
awk '/ERROR/{print NR ": " $0}' app.log
```

10. awk로 ERROR/WARN 선택

둘 중 하나가 있는 줄을 선택함.

```bash
awk '/ERROR|WARN/{print NR ": " $0}' app.log
```

11. 특정 줄 범위 출력

20~40줄만 편집기 없이 확인함.

```bash
awk 'NR>=20 && NR<=40 {print NR ": " $0}' test.jsp
```

12. JSP form 구간 출력

form 시작부터 종료 태그까지 출력함.

```bash
awk '/<form/,/<\/form>/{print NR ": " $0}' member.jsp
```

13. JSP script 구간 출력

JavaScript 구간을 추출함.

```bash
awk '/<script/,/<\/script>/{print NR ": " $0}' member.jsp
```

14. JSP 스크립틀릿 구간 출력

JSP <% ... %> 영역을 추출함.

```bash
awk '/<%/,/%>/{print NR ": " $0}' member.jsp
```

15. ERROR 개수 집계

로그 전체의 ERROR 건수를 계산함.

```bash
awk '/ERROR/{n++} END{print "ERROR:",n}' app.log
```

16. 기본 문자열 치환

파일은 바꾸지 않고 변경 결과만 화면에 미리 보여줌.

```bash
sed 's/8080/8081/g' application.properties
```

17. 실제 파일 치환

application.properties를 직접 수정함.

```bash
sed -i 's/8080/8081/g' application.properties
```

18. 백업 후 안전 치환

.bak 원본을 남긴 뒤 수정함.

```bash
sed -i.bak 's/8080/8081/g' application.properties
```

19. 특정 줄 삭제

10번째 줄을 삭제하며 백업을 남김.

```bash
sed -i.bak '10d' member.jsp
```

20. 줄 범위 삭제

10~20번째 줄을 삭제함.

```bash
sed -i.bak '10,20d' member.jsp
```

21. DEBUG 포함 줄 삭제

DEBUG 문자열이 있는 줄만 선택 삭제함.

```bash
sed -i.bak '/DEBUG/d' application.properties
```

22. 빈 줄 제거

빈 줄을 제거한 결과를 화면에 출력함.

```bash
sed '/^[[:space:]]*$/d' application.properties
```

23. 주석 줄 제거

#으로 시작하는 주석을 제외하고 출력함.

```bash
sed '/^[[:space:]]*#/d' application.properties
```

24. 특정 줄 뒤에 설정 삽입

server.port 줄 다음에 주소 설정을 삽입해 미리 확인함.

```bash
sed '/server.port/a server.address=0.0.0.0' application.properties
```

25. 특정 줄 앞에 주석 삽입

해당 설정 바로 앞에 설명 주석을 넣음.

```bash
sed '/server.port/i # Server Setting' application.properties
```

26. 파일 끝에 설정 추가

기존 파일 끝에 설정을 추가함.

```bash
echo "server.port=8080" | tee -a application.properties
```

27. root 설정파일에 추가

관리자 권한이 필요한 파일에 내용을 추가함.

```bash
echo "TEST=1" | sudo tee -a /etc/test.conf
```

28. 검색 결과 보고서 저장

화면에서 확인하면서 errors.txt에도 저장함.

```bash
grep -n "ERROR" app.log | tee errors.txt
```

29. awk 분석 결과 저장

```bash
awk 결과를 화면과 파일에 동시에 기록함.
awk '/ERROR/{print NR,$0}' app.log | tee errors.txt
```

30. JSP 전체에서 session 찾기

모든 JSP 파일의 파일명·줄 번호·구문을 출력함.

```bash
find . -name "*.jsp" -exec grep -nH "session" {} +
```

31. conf 전체에서 listen 찾기

conf 설정의 listen 위치를 찾음.

```bash
find /etc -name "*.conf" -exec grep -nH "listen" {} +
```

32. JSP 여러 파일 일괄 치환

모든 JSP에서 OLD를 NEW로 바꾸고 각 파일 백업을 남김.

```bash
find . -name "*.jsp" -exec sed -i.bak "s/OLD/NEW/g" {} +
```

33. properties에서 password 위치 검색

비밀번호 관련 설정의 위치를 찾음.

```bash
find . -name "*.properties" -exec grep -nH "password" {} +
```

34. TODO 보고서 만들기

미완료 코드 위치를 보고서로 저장함.

```bash
grep -Rni "TODO" src/ | tee todo.txt
```

35. IP 주소 사용 위치 검색

프로젝트에서 특정 IP가 사용된 위치를 찾음.

```bash
grep -Rni "192\.168\.0\.50" . | tee ip.txt
```

36. 여러 설정 확장자에서 localhost 검색

Spring 설정파일의 localhost 사용 위치를 검색함.

```bash
grep -Rni --include="*.yml" --include="*.yaml" --include="*.properties" "localhost" .
```

37. 수정 전후 검증

검색→백업→수정→재검색의 안전한 기본 패턴임.

```bash
grep -n '8080' application.properties | tee before.txt
sed -i.bak 's/8080/8081/g' application.properties
grep -n '8081' application.properties | tee after.txt
```

4. 자동화 Shell Script 예제 20개

1. JSP 에러 문자열 자동 검색

```bash
#!/bin/bash
DIR="/var/lib/tomcat/webapps"
REPORT="/tmp/jsp-error.txt"
grep -Rni --include="*.jsp" "error" "$DIR" | tee "$REPORT"
```

2. Java/Kotlin TODO 자동 검색

```bash
#!/bin/bash
grep -Rni --include="*.java" --include="*.kt" "TODO" src/ | tee todo-report.txt
```

3. Spring Boot 포트 자동 변경

```bash
#!/bin/bash
FILE="src/main/resources/application.properties"
cp "$FILE" "$FILE.bak"
sed -i 's/^server\.port=.*/server.port=8081/' "$FILE"
grep -n '^server\.port=' "$FILE" | tee port-result.txt
```

4. MariaDB 주소 자동 변경

```bash
#!/bin/bash
FILE="application.properties"
cp "$FILE" "$FILE.bak"
sed -i 's/192\.168\.0\.10/SERVER_IP/g' "$FILE"
grep -n "jdbc:mariadb" "$FILE" | tee database-result.txt
```

5. 설정파일 localhost 자동 검색

```bash
#!/bin/bash
grep -Rni --include="*.properties" --include="*.yml" --include="*.yaml" --include="*.conf" "lo
calhost" . | tee localhost-report.txt
```

6. ERROR/WARN 자동 분석

```bash
#!/bin/bash
LOG="app.log"
awk '
/ERROR/ {print "[ERROR]", NR, $0}
/WARN/  {print "[WARN ]", NR, $0}
' "$LOG" | tee error-warn-report.txt
```

7. ERROR/WARN 건수 자동 통계

```bash
#!/bin/bash
awk '
/ERROR/ {error++}
/WARN/  {warn++}
END {
  print "ERROR =", error+0
  print "WARN  =", warn+0
}
' app.log | tee log-statistics.txt
```

8. JSP DEBUG 코드 자동 검색

```bash
#!/bin/bash
find . -type f -name "*.jsp" -exec grep -niH "DEBUG" {} + | tee jsp-debug.txt
```

9. JSP DEBUG 줄 백업 후 삭제

```bash
#!/bin/bash
find . -type f -name "*.jsp" -exec sed -i.bak '/DEBUG/d' {} +
echo "DEBUG 제거 완료" | tee debug-delete.log
```

10. properties 자동 백업

```bash
#!/bin/bash
DATE=$(date +%Y%m%d-%H%M%S)
BACKUP="backup-$DATE"
mkdir -p "$BACKUP"
find . -type f -name "*.properties" -exec cp --parents {} "$BACKUP" \;
echo "Backup: $BACKUP" | tee backup.log
```

11. 설정파일 전체 자동 백업

```bash
#!/bin/bash
BACKUP="config-$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP"
find . -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" -o -name "*.conf" \)
-exec cp --parents {} "$BACKUP" \;
echo "$BACKUP" | tee backup-result.txt
```

12. localhost 일괄 변경

```bash
#!/bin/bash
find . -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" \)   -exec sed -i.ba
k 's/localhost/SERVER_IP/g' {} +
grep -Rni "SERVER_IP" . | tee replace-result.txt
```

13. JSP form 코드 자동 추출

```bash
#!/bin/bash
FILE="member.jsp"
awk '/<form/,/<\/form>/{print NR ": " $0}' "$FILE" | tee form-code.txt
```

14. JSP 스크립틀릿 자동 추출

```bash
#!/bin/bash
FILE="member.jsp"
awk '/<%/,/%>/{print NR ": " $0}' "$FILE" | tee jsp-java-code.txt
```

15. Spring Boot 설정 자동 검사

```bash
#!/bin/bash
FILE="application.properties"
{
 echo "===== PORT ====="
 grep -n '^server\.port' "$FILE"
 echo "===== DATABASE ====="
 grep -n 'datasource' "$FILE"
 echo "===== JPA ====="
 grep -n 'jpa' "$FILE"
} | tee config-report.txt
```

16. MariaDB 설정 위치 자동 추출

```bash
#!/bin/bash
grep -Rni --include="*.properties" --include="*.yml" --include="*.yaml" "mariadb" . | tee mari
adb-config.txt
```

17. 비밀번호 설정 파일 위치만 검사

```bash
#!/bin/bash
grep -Rnil --include="*.properties" --include="*.yml" --include="*.yaml" "password" . | tee pa
ssword-files.txt
```

18. Java/Kotlin/JSP 통합 코드 검색

```bash
#!/bin/bash
SEARCH="$1"
if [ -z "$SEARCH" ]; then
  echo "사용법: $0 검색어"
  exit 1
fi
grep -Rni --include="*.java" --include="*.kt" --include="*.jsp" "$SEARCH" . | tee search-result.txt
```

19. 서버 로그 일일 자동 분석

```bash
#!/bin/bash
LOG="/home/devuser/app/app.log"
DATE=$(date +%Y%m%d)
REPORT="/home/devuser/report-$DATE.txt"
{
 echo "===== SERVER REPORT ====="
 date
 echo "===== ERROR ====="
 grep -ni "ERROR" "$LOG" | tail -50
 echo "===== WARN ====="
 grep -ni "WARN" "$LOG" | tail -50
 echo "===== COUNT ====="
 awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR:",e+0; print "WARN:",w+0}' "$LOG"
} | tee "$REPORT"
```

20. 프로젝트 통합 자동 검사

```bash
#!/bin/bash
PROJECT="/home/devuser/app"
REPORT="$PROJECT/project-report.txt"
cd "$PROJECT" || exit 1
{
 echo "================================"
 echo "PROJECT CHECK"
 date
 echo "===== JAVA PROCESS ====="
 pgrep -af java || echo "Java process 없음"
 echo "===== PORT 8080 ====="
 ss -tulpn | grep 8080 || echo "8080 port 없음"
 echo "===== ERROR ====="
 grep -ni "ERROR" app.log | tail -20
 echo "===== WARN ====="
 grep -ni "WARN" app.log | tail -20
 echo "===== ERROR/WARN COUNT ====="
 awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR:",e+0; print "WARN:",w+0}' app.log
 echo "===== SERVER PORT ====="
 grep -Rni --include="*.properties" --include="*.yml" "server.port" .
 echo "===== DATABASE ====="
 grep -Rni --include="*.properties" --include="*.yml" "datasource" .
 echo "===== TODO ====="
 grep -Rni --include="*.java" --include="*.kt" --include="*.jsp" "TODO" .
 echo "===== DISK ====="
 df -h
 echo "===== MEMORY ====="
 free -h
} | tee -a "$REPORT"
```

5. cron 자동 실행 예

스크립트에 실행 권한을 부여한 뒤 crontab -e에서 등록함.

```bash
chmod +x project-check.sh
0 9 * * * /home/devuser/project-check.sh
0 23 * * * /home/devuser/log-report.sh
```

첫 번째 예는 매일 오전 9시, 두 번째 예는 매일 오후 11시에 실행함.

6. 실무 안전 원칙

- sed -i 또는 find와 sed를 결합한 일괄 수정 전에 grep으로 변경 대상을 먼저 확인함.
- 가능하면 sed -i.bak 또는 별도 백업 디렉터리를 사용하여 원본을 보존함.
- 자동 삭제는 검색 보고서를 먼저 만든 뒤 사람이 확인하고 적용하는 2단계 방식이 안전함.
- password, API key 등 비밀정보는 tee 보고서에 실제 값이 남지 않도록 파일명·위치만 검사하는 것이 좋음.
- 운영 서버의 Spring Boot 프로세스 자동 재시작은 단순 cron보다 systemd의 Restart 정책을 우선 검토함.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.